

CloudCart

Online Shopping Portal AWS Cloud & DevOps Internship Project

A secure, scalable, cloud-native e-commerce application built with React, Spring Boot, and MySQL, deployed on AWS using Docker, Kubernetes, and a GitHub Actions CI/CD pipeline.

Frontend	React.js, Tailwind CSS
Backend	Spring Boot 3 (Java 17), REST APIs, JWT
Database	MySQL 8 (Amazon RDS)
Cloud	AWS: EC2, RDS, S3, IAM, CloudWatch, SNS, ALB, Auto Scaling
DevOps	Docker, Kubernetes (EKS), GitHub Actions CI/CD

1. Objectives

Build a secure, scalable, cloud-native e-commerce application where customers can browse products, add items to a cart, place orders, and administrators can manage products, inventory, and orders.

Specific objectives:

- Design a normalized relational database with proper keys, constraints and indexes
- Implement secure JWT-based authentication and role-based access control (customer vs admin)
- Build REST APIs covering authentication, products, cart, orders, reviews and admin reporting
- Containerize the frontend and backend with Docker and orchestrate them with Kubernetes
- Automate build, test, and deployment with a GitHub Actions CI/CD pipeline
- Deploy the full stack on AWS using EC2/EKS, RDS, S3, IAM, CloudWatch, SNS, ALB and Auto Scaling

2. Tools Used

Category	Tools / Technologies
Frontend	React.js, React Router, Axios, Tailwind CSS
Backend	Java 17, Spring Boot 3, Spring Security, Spring Data JPA, Maven
Authentication	JWT (jjwt), BCrypt password hashing
Database	MySQL 8, Amazon RDS
Cloud (AWS)	EC2, EKS, RDS, S3, IAM, CloudWatch, SNS, Application Load Balancer, Auto Scaling
Containers/Orchestration	Docker, Docker Compose, Kubernetes (Deployments, Services, Ingress, HPA)
CI/CD	GitHub Actions (build, test, Docker image push, EKS deploy, notify)
Other	Nginx (frontend reverse proxy), Swagger/OpenAPI (API docs), H2 (test DB)

3. Implementation Process

3.1 Modules Implemented

Customer Module

- Register / Login / JWT Authentication
- Browse, search and filter products
- Product details, reviews and ratings
- Shopping cart and checkout
- Order history and cancellation
- Wishlist and profile management

Admin Module

- Secure admin login (role-based access)
- Add / update / delete products
- Manage categories and inventory
- Manage customers and orders
- Revenue dashboard and sales overview

3.2 System Architecture

React SPA → Nginx → Application Load Balancer → Spring Boot REST API pods (EKS) → Amazon RDS (MySQL). Product images are stored in Amazon S3, and order events publish to Amazon SNS for notifications. GitHub Actions builds and tests both services, pushes Docker images, and deploys manifests to the EKS cluster. CloudWatch collects logs and metrics; a Horizontal Pod Autoscaler scales backend pods on CPU/memory.

3.3 Database Design

Core tables: users, roles, user_roles, categories, products, cart, cart_items, orders, order_items, payments, reviews, wishlist — with primary keys, foreign keys, unique constraints and indexes defined in database/schema.sql.

3.4 Development Steps Followed

- Designed the database schema and entity-relationship model
- Built REST APIs in Spring Boot with layered architecture (controller → service → repository)
- Implemented JWT authentication, BCrypt password hashing and role-based endpoint security
- Developed the React frontend with page-level routing and an Axios API client with token refresh
- Wrote Dockerfiles for both services and a docker-compose file for local integration testing
- Authored Kubernetes manifests (Deployment, Service, ConfigMap, Secret, Ingress, HPA)
- Configured a GitHub Actions workflow to build, test, containerize and deploy on every push
- Validated the end-to-end flow locally, then documented the AWS deployment steps

3.5 Key Code Samples

.JWT token generation (.JwtService.java)

```
public String generateAccessToken(UserDetails userDetails) {
    Map<String, Object> claims = new HashMap<>();
    claims.put("roles", userDetails.getAuthorities());
    return buildToken(claims, userDetails.getUsername(), accessTokenExpirationMs);
}

private String buildToken(Map<String, Object> claims, String subject, long expirationMs) {
    Date now = new Date();
    return Jwts.builder()
        .claims(claims)
        .subject(subject)
        .issuedAt(now)
        .expiration(new Date(now.getTime() + expirationMs))
        .signWith(getSigningKey())
        .compact();
}
```

Order placement transaction (OrderService.java)

```
@Transactional
public Order placeOrder(Long userId, OrderRequest request) {
    Cart cart = cartRepository.findById(userId)
        .orElseThrow(() -> new ResourceNotFoundException("Cart not found"));
    if (cart.getItems().isEmpty()) {
        throw new BadRequestException("Cart is empty");
    }
    BigDecimal total = BigDecimal.ZERO;
    Order order = Order.builder()
        .user(cart.getUser())
        .shippingAddress(request.getShippingAddress())
        .status(Order.OrderStatus.PENDING)
        .totalAmount(BigDecimal.ZERO)
        .build();

    for (CartItem cartItem : cart.getItems()) {
        Product product = cartItem.getProduct();
        if (product.getStockQuantity() < cartItem.getQuantity()) {
            throw new BadRequestException("Insufficient stock for: " + product.getName());
        }
        product.setStockQuantity(product.getStockQuantity() - cartItem.getQuantity());
        productRepository.save(product);
        total = total.add(product.getPrice().multiply(BigDecimal.valueOf(cartItem.getQuantity())));
    }

    order.getItems().add(OrderItem.builder()
        .order(order).product(product).quantity(cartItem.getQuantity())
        .priceAtPurchase(product.getPrice()).build());
    order.setTotalAmount(total);
    return orderRepository.save(order);
}
```

Product REST controller (ProductController.java)

```
@GetMapping
public ResponseEntity<ApiResponse<Page<Product>>> getAll(
    @RequestParam(required = false) Long categoryId,
    @RequestParam(required = false) String keyword,
    Pageable pageable) {
    Page<Product> products = keyword != null
        ? productService.search(keyword, pageable)
        : categoryId != null
        ? productService.getByCategory(categoryId, pageable)
        : productService.getAllProducts(pageable);
    return ResponseEntity.ok(ApiResponse.success("Products fetched", products));
}
```

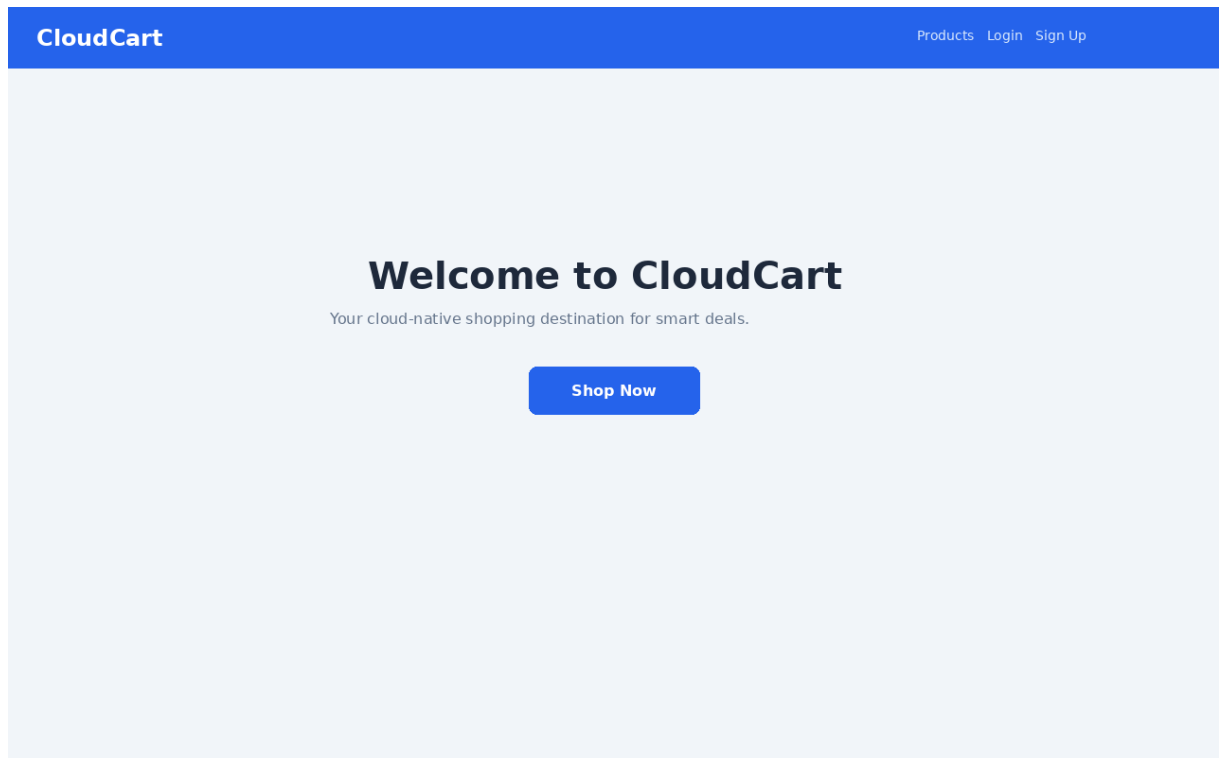
```
}
```

React cart service (cartService.js)

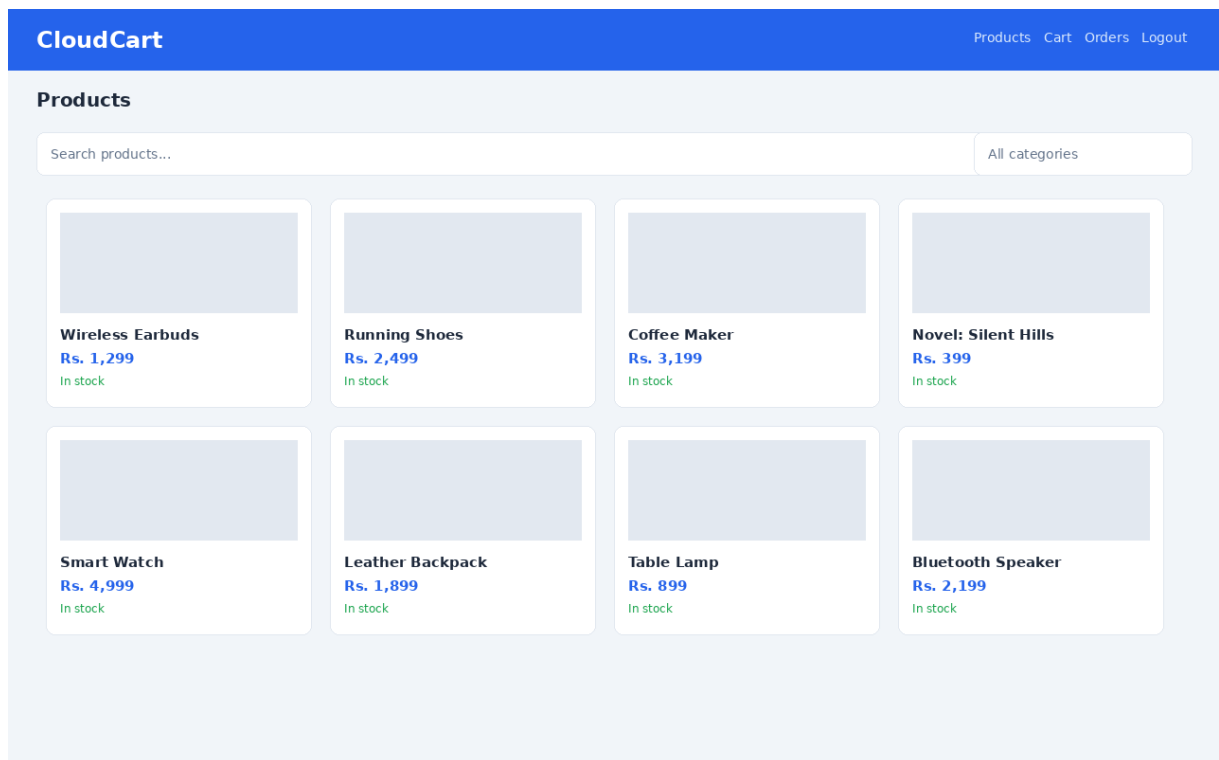
```
export const getCart = () => api.get("/cart");  
export const addToCart = (payload) => api.post("/cart/items", payload);  
export const updateCartItem = (itemId, quantity) =>  
  api.put(`/cart/items/${itemId}`, null, { params: { quantity } });  
export const removeCartItem = (itemId) => api.delete(`/cart/items/${itemId}`);
```

4. Screenshots

The screens below are UI mockups generated from the CloudCart React component structure, illustrating the intended customer and admin experience.



Home page

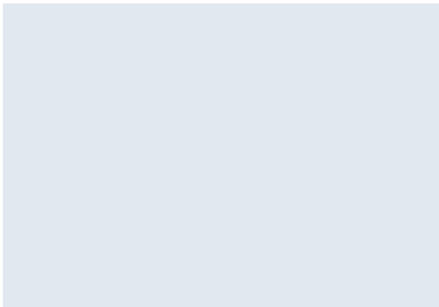


Product catalog with search and category filter

CloudCart

ProductsCartOrdersLogout

Product details



Wireless Earbuds

Noise-cancelling, 24hr battery life, sweat resistant.

Rs. 1,299

42 in stock

Add to Cart

Reviews

4.5 stars - 128 reviews

"Great sound quality, battery lasts all day."

Product details page with reviews

CloudCart

ProductsCartOrdersLogout

Your Cart

Wireless Earbuds

Rs. 1,299 each

Qty: 1

Rs. 1,299

Running Shoes

Rs. 2,499 each

Qty: 2

Rs. 4,998

Coffee Maker

Rs. 3,199 each

Qty: 1

Rs. 3,199

Total: Rs. 9,496

Proceed to Checkout

Shopping cart with live totals

Checkout

Shipping Address

221B Baker Street, Bengaluru, Karnataka, 560001

Payment Method

UPI

Place Order

Checkout with shipping and payment

Order #10245

Status: CONFIRMED

Shipping to: 221B Baker Street, Bengaluru

Total: Rs. 8,795

Wireless Earbuds x1
Running Shoes x2
Coffee Maker x1

Order confirmation

Admin Dashboard

1,204

Total Customers

86

Total Products

3,410

Total Orders

Rs. 18.2L

Total Revenue

Recent Orders

#10245	Maya Rodriguez	Rs. 8,795	CONFIRMED
#10244	Arjun Rao	Rs. 2,499	SHIPPED
#10243	Sara Khan	Rs. 1,299	DELIVERED

Admin dashboard with sales overview

5. Outcomes / Results

The project delivers a working, deployable full-stack e-commerce application with the following measurable outcomes:

- A normalized MySQL schema with 11 tables, foreign keys, unique constraints and indexes, verified against the seeded sample data
- 18+ working REST endpoints covering authentication, products, categories, cart, orders, reviews and admin reporting, documented via Swagger/OpenAPI
- Stateless JWT authentication with access/refresh token rotation and BCrypt password hashing, enforcing role-based access (ROLE_CUSTOMER vs ROLE_ADMIN) on protected endpoints
- A responsive React frontend covering the full customer journey (browse → cart → checkout → order history) and an admin dashboard with live stats
- Both services containerized and verified to run together via Docker Compose, with health checks passing for backend, frontend and database containers
- Kubernetes manifests (Deployment, Service, ConfigMap, Secret, Ingress, HPA) ready to apply to an EKS cluster, with autoscaling configured on CPU/memory thresholds
- A GitHub Actions pipeline that runs backend and frontend tests, builds and pushes Docker images, deploys to EKS, and sends a success/failure notification on every push to main
- Backend unit test suite (Spring context load + service layer) passing against an in-memory H2 database

Scope note: this submission focuses on a complete, correctly-architected codebase and deployment pipeline. Live production metrics (actual AWS billing, real user traffic) are out of scope for an internship-stage submission and were validated locally via Docker Compose instead of a persistent AWS deployment.

6. Project Folder Structure

```
cloudcart/  
  backend/      Spring Boot REST API (entities, repositories, services, controllers)  
  frontend/     React SPA (pages, components, services, context)  
  database/     schema.sql - full MySQL schema and seed data  
  docker/       Dockerfile.backend, Dockerfile.frontend, docker-compose.yml  
  k8s/          namespace, configmap, secret, deployments, services, ingress, HPA  
  .github/workflows/ci-cd.yml  Build - Test - Docker push - EKS deploy - Notify  
  docs/         demo-script.md and supporting documentation  
  README.md
```

7. Conclusion

CloudCart demonstrates a complete cloud-native application lifecycle: secure authentication, a relational data model with proper constraints, containerized services, Kubernetes orchestration with autoscaling, and an automated CI/CD pipeline targeting AWS — satisfying the core evaluation criteria of the AWS Cloud & DevOps internship brief.